

Artificial Intelligence

Shover Project Phase 2

Farzaneh Khalili – Hasti Javadzadeh

Introduction

The objective of Phase Two of this project was to design and implement an intelligent Player AI capable of solving Shover World challenges using classical search methods enhanced with domain-specific reasoning. Phase Two emphasized efficient state-space search, heuristic design, and scalability to larger maps, specifically up to the required size of 9×13. The phase also required creativity in state representation, heuristic functions, and goal decomposition to ensure tractable performance.

State Representation Design

A critical early decision in Phase Two was the definition of the internal state used by the search algorithm. The chosen state representation includes the agent's position, the agent's facing direction, and the set of all remaining box positions. This representation was implemented using an immutable dataclass to ensure hashability and safe use in closed sets during search.

The inclusion of the agent's position was essential for correctness, as reachability and distance to boxes directly affect future actions. The agent's direction was also retained because action validity and outcomes depend on orientation. The set of boxes was stored as a frozen set to allow efficient comparison and hashing.

Other environment attributes such as stamina were intentionally excluded from the state representation. Instead of expanding the state space, stamina was incorporated into heuristic evaluation, allowing the search to remain focused while still being energy-aware.

The final state representation achieved a balance between completeness and compactness, enabling correct simulation while avoiding unnecessary state explosion.

Environment–State Conversion Interface

To integrate the search algorithm with the Shover World simulator, a API was designed to convert between environment states and internal search states. The function `env_to_state` extracts relevant information from the environment grid and agent configuration to construct a

search state. Conversely, `state_to_env` reconstructs a fresh environment instance from a given search state and a template environment.

Successor Generation and Cost Modeling

Successor generation was implemented by reconstructing the environment for each state and applying all possible actions. Invalid actions were filtered using the environment's feedback. Each valid action produced a successor state along with an associated cost.

Instead of using a uniform step cost, the cost function was defined as the negative of the environment reward. This aligned the A* search objective with the game's scoring system, encouraging box destruction and discouraging wasteful movement. This design allowed the search to naturally prefer solutions that destroy boxes efficiently and conserve resources.

Baseline Heuristic Design and Evaluation

Several baseline heuristics were implemented and experimentally evaluated. The simplest heuristic combined the number of remaining boxes with the Manhattan distance from the agent to the nearest box. More advanced variants computed the total Manhattan distance from each box to the nearest lava cell, optionally combined with the agent-to-box distance.

These heuristics were admissible and effective on small maps but showed rapid degradation as map size and box count increased. Experimental results on 9×13 maps demonstrated large numbers of expanded nodes and open-list sizes, indicating insufficient guidance in complex environments.

Energy Heuristic

We implemented the `EnergyHeuristic` class to guide the A* search process in the Shover World environment. We reconstruct a full environment state from a compact planning state and evaluate progress toward the goal using multiple features, including the number of remaining boxes, agent stamina, distance to lava, chain length of adjacent boxes, and the presence of perfect square patterns. These features are combined into a weighted heuristic that favors states closer to eliminating all boxes. To ensure correctness, we safely handle perfect square detection by avoiding mutations of the grid or environment, preventing unintended side effects during planning.

In addition to the base heuristic, we applied subgoal-based shaping to bias the search toward intermediate objectives such as approaching a box, pushing a box into lava, or exploiting perfect square configurations. This approach improves search efficiency by focusing exploration on locally relevant actions, even though it relies on approximate distance measures that do not always reflect true feasibility. As a result, the heuristic is not admissible and may produce longer plans in exchange for faster search. Overall, we prioritized practical performance and reliable problem solving over strict optimality guarantees.

Subgoal Manager

We designed the `SubgoalManager` class to generate high-level intermediate objectives that guide planning in the Shover World environment. Given a state representation, we produce a prioritized list of subgoals based on the current grid configuration: first pushing any box adjacent to lava into lava, then exploiting detected perfect square patterns if available, and finally approaching the nearest remaining box as a fallback strategy. To ensure that subgoals accurately reflect the current planning state, we reconstruct an environment snapshot using `state_to_env` and select the highest-priority subgoal from the resulting env-like representation.

We also implemented an `is_subgoal_reached` function to determine whether a selected subgoal has been achieved. This function supports both env-like dictionary states and the planner's `State` dataclass by converting states when necessary. Additional helper functions implement the core detection logic: we identify boxes adjacent to lava, compute the nearest box using Manhattan distance, and detect perfect square structures through the environment's perfect square manager. Together, these components provide structured guidance to the planner while remaining lightweight and flexible.

Gui_ai

We implemented a graphical user interface using Pygame to visualize and interact with the Shover World environment, supporting both manual control and AI-driven execution. The GUI initializes the environment, renders the grid, agent, obstacles, lava, and special structures, and continuously processes user input through an event loop. Manual controls allow us to move the agent, place barriers, reset the environment, or directly manipulate the agent for debugging. In addition, the interface displays key runtime information such as timestep, stamina, agent position, total reward, and planning progress, providing clear feedback during execution.

We integrated the A* planner into the GUI through a plan provider function that computes a sequence of actions from the current state and executes them step by step in AI mode. By toggling AI control, we were able to observe how planned actions are carried out in the real environment and verify the correctness of the planner through replay. The system includes safeguards to stop execution if invalid actions occur and allows pausing or resuming AI behavior.

